

CodeSheriff: Design and Development of an Agentic Security Auditor for an SFI-Enabled Confidential CI/CD Execution Layer Using Bayesian Vulnerability Inference

Shubhankar Vishnu Patil

Dept. of Information Technology

G H Raisoni College of Engineering and Management

Pune, India

shubhankarp258@gmail.com

Vivek Dnyaneshwar Joshi

Dept. of Information Technology

G H Raisoni College of Engineering and Management

Pune, India

vivekdjoshi05@gmail.com

Yash Suresh Kodgirwar

Dept. of Information Technology

G H Raisoni College of Engineering and Management

Pune, India

yashkodgirwar@gmail.com

Nirmalya Mukhopadhyay

Dept. of Information Technology

G H Raisoni College of Engineering and Management

Pune, India

Kunal Kundu

Dept. of Information Technology

G H Raisoni College of Engineering and Management

Pune, India

Abstract—Modern software development increasingly relies on Continuous Integration/Continuous Deployment (CI/CD) pipelines to deliver software rapidly, but automated security review at the pull request stage remains limited. Existing security tools rely on rigid static analysis or homogeneous multi-agent approaches, which may generate false positives or overlook complex vulnerabilities, and they report a verdict without any measure of how often that verdict is right. This paper presents CodeSheriff, an agentic security auditor that analyzes GitHub pull requests using structural, semantic, contextual, and SFI-based confidential execution analysis. A Bayesian inference engine fuses evidence from the four agents into a calibrated vulnerability probability, and an automated patch generation and verification module recommends and validates fixes through GitHub and a web dashboard. On a labelled corpus of 76 Python functions arranged as 38 vulnerable–safe twin pairs, CodeSheriff attains an F_1 of 0.923 at a precision of 1.000 with an expected calibration error of 0.027, while majority voting over identical agent evidence attains an F_1 of only 0.142 at five times the calibration error. The system therefore delivers reliable, explainable, and statistically grounded vulnerability detection while reducing manual review effort.

Index Terms—vulnerability detection, multi-agent systems, software security, Bayesian inference, probability calibration, software fault isolation, continuous integration and continuous deployment

I. INTRODUCTION

In today’s software industry, developers frequently push code to the main branch through Pull Requests (PRs). However, security analysis of each Pull Request is still largely performed manually, making the review process more time-consuming. Teams either use basic approaches that only catch

known, simple mistakes, or they don’t have enough time to properly review every pull request. This is a big problem because even one missed vulnerability can put the whole application at risk once the code is merged and deployed. On top of this, attackers are now also targeting the Continuous Integration/Continuous Deployment (CI/CD) that’s builds and runs the code.

Solutions that exist today don’t fully solve this problem. Traditional static analysis approaches identify fixed patterns but are unable to catch tricky ones, so they generate a lot of false alarms. Some recent tools use multiple artificial intelligence (AI) agents, but all these agents basically read the same code in the same way, so they don’t really give different opinions. When these approaches combine results, they just use simple voting, which doesn’t tell you how confident the final answer actually is.

In this paper, we are proposing CodeSheriff to fix this problem. It uses four different types of agents that look at the code from different perspectives. Static agent checks the code structure, the meaning of the code is understood by the semantic agent, the context agent checks whether accepting this PR could bring back any vulnerability from previously accepted PR, and the runtime agent actually watches the code run inside a safe, isolated environment using software fault isolation (SFI). All this information is then combined using a Bayesian model, which gives a clear probability of whether the code is vulnerable, along with how confident the system is. This same safe environment also protects the CI/CD workflow, since it only releases secret keys when it is sure the correct, verified code is running.

The rest of this paper is structured as follows. Section II talks about related work already done in this area. Section III explains our proposed method, its mathematical model, algorithms and workflow. Section IV describes the implementation. Section V reports and discusses the results, and Section VI concludes the paper with the future scope.

II. LITERATURE SURVEY

Early learning-based studies explored structural and sequential representations for vulnerability detection. Li and Yang [1] compared sequence and graph-based deep learning methods, identifying challenges in capturing long-range dependencies and generalizing to noisy data. CODE-SMASH [2] and XTransformer [3] employed hierarchical neural and transformer architectures to capture multi-level code representations and improve similarity-based detection, while remaining sensitive to computational cost and dataset characteristics. Abbasi and Shameli-Sendi [4] used Code Property Graph metrics with machine learning for interpretable vulnerability and vulnerability-type prediction, but their approach was limited to C/C++. DeepSecure [5] combined CodeBERT with bidirectional long short-term memory (BiLSTM) and convolutional neural network (CNN) layers for python vulnerability detection, incorporating robustness and confidence calibration, but remained language-specific and affected by noisy labels.

Transformer and large language model (LLM) based approaches have subsequently improved semantic analysis of source code. Shestov *et al.* [6] fine-tuned WizardCoder using parameter-efficient learning for vulnerability classification, but remained limited to function-level analysis. Tamberg and Bahsi and Gnieciak and Szandala [7], [8] benchmarked LLMs against vulnerability detection tasks and traditional static analysis tools, showing strong potential but persistent false positives, hallucinations, localization errors, and inference costs. Gülmez [9] surveyed code-generation LLMs and highlighted challenges in structural correctness, repository-level reasoning, and security. Chen *et al.* [10] and Su *et al.* [11] similarly identified limitations involving context windows, hallucination, dataset quality, generalization, and interpretability. Germano *et al.* [12] reviewed LLM-based vulnerability detection, repair, and explanation, noting that research remains more focused on detection than automated repair and explanation.

To address the limitations of isolated code analysis, recent studies have incorporated contextual and repository-level information. Antal *et al.* [13] evaluated retrieval augmented generation (RAG) for real-world Java vulnerability detection using retrieved vulnerable and secure examples, but observed false positives and retrieval sparsity. AegisGuard [14] incorporated contextual information for semantic vulnerability detection and risk stratification, while SCOUT [15] enabled interactive repository exploration to retrieve relevant code context. Göçmen *et al.* [16] integrated repository history and dependency analysis into pull-request reviews for change-impact estimation, although static dependency analysis cannot fully capture dynamic behavior.

Multi-agent approaches have further introduced collaborative security reasoning. Wei *et al.* [17] used specialized roles and collaborative reasoning for vulnerability auditing, while MAVUL [18] combined vulnerability analysis, security architecture review, and iterative evaluation. Reddy *et al.* [19] explored knowledge-based agentic AI for automated vulnerability triage, and Syed *et al.* [20] investigated an observe–reason–act–verify framework for autonomous software-supply-chain defense. These studies demonstrate the potential of agentic security workflows, while also highlighting challenges such as reduced performance on complex cases, token overhead, and the need for human intervention.

However, existing approaches do not effectively combine structural, semantic and contextual analysis within a unified vulnerability detection framework. We integrate these approaches through specialized agents and confidence-aware vulnerability inference, while providing explanations and patch recommendations within the GitHub pull-request workflow.

III. PROPOSED METHODOLOGY

A vulnerability can hide in the structure of the code, in its intent, in a departure from the repository’s own habits, or only in what the code does when it runs. CodeSheriff therefore analyses every changed function with four independent agents – Structural, Semantic, Context and Runtime – that never see each other’s output, and fuses their evidence by Bayesian inference instead of voting, so the reported confidence is calibrated rather than asserted.

A. Mathematical Model

The framework evaluates a pull request in five stages: change extraction, independent agent analysis, evidence categorisation, Bayesian fusion and the alert decision. The unit of analysis is one changed function u , obtained by mapping every line that differs between the base and head versions of a file to the outermost function containing it.

1) *Structural Agent*: The Structural Agent follows data through a definition–use graph from each function parameter, which is treated as untrusted, to a security-sensitive operation (a *sink*) such as `cursor.execute` or `os.system`, and checks whether the path is sanitized. When an unsanitized path reaches a sink, the structural score is

$$S_{\text{str}} = \text{clamp}_{[0,1]}(0.45 + \delta + 0.15\nu - 0.25\varsigma - 0.20\tau - \lambda) \quad (1)$$

where δ is 0.20 for a critical sink and 0.10 for a high-severity sink, $\nu = 1$ for a network-facing unit, $\varsigma = 1$ when a partial sanitizer lies on the path, $\tau = 1$ in a test file, and $\lambda \leq 0.05$ is a small penalty for long paths. Semgrep runs as a second backend on the same unit, and the larger of the two scores is kept, because both read the same source text with the same technique.

2) *Semantic Agent*: The Semantic Agent uses a large language model to infer the functional intent of the code, its trust boundaries and the security invariant that the change breaks. The model is sampled $n = 3$ times with different seeds, and a sample is accepted only if the weakness class is in scope, the

file matches, the quoted sink appears verbatim in the source and the reported lines lie inside the unit. For weakness class c , the semantic score is the agreement across accepted samples:

$$S_{\text{sem}}(c) = \frac{m_c}{n} \quad (2)$$

where m_c is the number of accepted samples reporting c and n is the total number of samples. A single sample cannot separate confidence from chance, which is why agreement, not the model's own wording, is the score.

3) *Context Agent*: The Context Agent retrieves the repository's own merged history using local sentence embeddings stored in pgvector and reports a security control γ that the history applies and the changed function does not. The contextual score is

$$S_{\text{ctx}}(\gamma) = b \cdot \min\left(1, \frac{\bar{s}}{0.85}\right) \quad (3)$$

where $\bar{s}$ is the mean similarity of the retrieved supporting functions, and $b = 0.85$ when the same function established γ in an earlier merge or three or more sibling functions apply it, and $b = 0.65$ when exactly two siblings apply it. A single neighbour is treated as coincidence and reports nothing.

4) *Runtime Agent*: The Runtime Agent executes the changed function once inside a WebAssembly sandbox built on Wasmtime and WASI, which enforces software fault isolation at the instruction level: the network is denied at link time, the environment is empty and fuel, time and memory are capped. Each parameter is bound to a unique random token, so a value is untrusted exactly when one of those tokens appears inside it. If such a value reaches a sink, the agent reports a detection with the fixed score $S_{\text{rt}} = 0.85$, a constant because this is a direct observation and not an estimate. If the sink was reached only because the sandbox had to answer a guard it could not evaluate, the agent abstains.

5) *Bayesian Fusion and Decision*: Every agent returns a *detection* with a score, a *silence* (it ran and found nothing) or an *abstention* (it could not run). Raw scores from different agents are not directly comparable, so each statement is first mapped to one shared evidence cell:

$$z_a = \begin{cases} \text{HIGH}, & S_a \geq 0.8 \\ \text{MEDIUM}, & 0.5 \leq S_a < 0.8 \\ \text{LOW}, & S_a < 0.5 \\ \text{SILENCE}, & \text{agent ran, found nothing} \end{cases} \quad (4)$$

Each cell is weighted by how reliably it has predicted vulnerability in labelled data, through a likelihood ratio estimated with Laplace smoothing:

$$\Lambda_a(z) = \frac{P(z | V)}{P(z | \neg V)} \approx \frac{(k_V + 1)/(N_V + 4)}{(k_S + 1)/(N_S + 4)} \quad (5)$$

where k_V and k_S are the numbers of vulnerable and safe examples that fell into cell z , and N_V and N_S are the corresponding totals. An abstention, and any cell never observed, takes $\Lambda_a = 1$, so an agent that could not look leaves the result unchanged instead of being read as a vote of "safe". Ratios are clamped to $[0.05, 20]$ so that no single agent can dominate.

The prior odds are then updated multiplicatively and converted back into a probability:

$$P(V | E) = \frac{O}{1 + O}, \quad O = \frac{\pi}{1 - \pi} \prod_{a \in \{S, M, C, R\}} \Lambda_a(z_a) \quad (6)$$

where π is the prior probability of a vulnerability and O is the posterior odds. The product always contains four factors, one per agent, and a silence contributes a factor below 1, so the probability can fall as well as rise. The finding is finally labelled

$$\text{Verdict} = \begin{cases} \text{ALERT}, & P(V | E) \geq \tau \\ \text{NO ALERT}, & \text{otherwise} \end{cases} \quad (7)$$

where τ is the alert threshold fitted on labelled data. With $\pi = 0.03$ and $\tau = 0.227$, one strong semantic detection that no other agent contradicts is exactly sufficient to alert, while a medium structural, a high semantic and a high runtime detection together give $P(V | E) \approx 0.97$.

B. Algorithms

The framework is implemented as two algorithmic stages: independent evidence generation by the four agents, and Bayesian fusion with the alert decision and patch handoff.

Algorithm 1: Agent Evidence Generation

Input: Changed function u
Output: Evidence $E = \{e_S, e_M, e_C, e_R\}$
1 **foreach** agent $a \in \{S, M, C, R\}$ **do**
2 **if** a cannot analyse u **then**
3 $e_a \leftarrow \text{ABSTENTION}(\text{reason});$
4 **else**
5 $F_a \leftarrow$ findings of a on u ; // Eqs. (1) -- (3)
6 **if** $F_a \neq \emptyset$ **then** $e_a \leftarrow \text{DETECTION}(S_a, c);$
7 **else** $e_a \leftarrow \text{SILENCE};$
8 **return** $E;$

Algorithm 1 runs the four agents independently on one changed function. Every agent always returns a statement, so a failure is recorded as an abstention and is never confused with "analysed and found nothing".

Algorithm 2 multiplies the prior odds by one fitted ratio per agent, converts the result into a probability and compares it with the threshold. An alerted finding is passed to the patch stage, where a drafted repair must parse, keep the function signature, use only names the file defines, change the code, and survive re-examination by the Structural and Runtime agents before it is suggested. CodeSheriff only suggests; it never commits or merges.

Algorithm 2: Bayesian Fusion, Decision and Patching

Input: Evidence E , ratios Λ , prior π , threshold τ
Output: $P(V | E)$, Verdict Y , patch F

```

1  $O \leftarrow \pi / (1 - \pi)$ ;
2 foreach agent  $a$  do
3    $z_a \leftarrow \text{cell of } e_a$ ; // Eq. (4)
4    $O \leftarrow O \times \Lambda_a(z_a)$ ; //  $\Lambda_a = 1$  if abstained
5  $P(V | E) \leftarrow O / (1 + O)$ ;
6  $Y \leftarrow \text{ALERT}$  if  $P(V | E) \geq \tau$  else NO ALERT;
7 if  $Y = \text{ALERT}$  then
8    $F \leftarrow \text{DRAFTPATCH}(u, E)$ , verified by the deterministic agents;
9   publish  $F$  as a suggestion only if it lies inside the changed lines;
10 else  $F \leftarrow \emptyset$ ;
11 return  $P(V | E)$ ,  $Y$ ,  $F$ ;
```

C. System Workflow

The proposed solution is integrated into the GitHub PRs workflow to perform automated security analysis with minimal manual intervention. When a developer opens a Pull Request, a GitHub webhook triggers the orchestrator, which collects the code changes. The changed code is then processed through diff extraction, parsing, and normalization to prepare it for further analysis. Specifically, this work makes three contributions: a heterogeneous four-agent framework that reasons about a pull request from structural, semantic, historical, and execution-based perspectives; a Bayesian fusion mechanism that produces a calibrated, confidence-aware vulnerability probability rather than a binary alert; and an automated patch generation and verification pipeline that proposes and validates fixes directly within the GitHub pull request workflow.

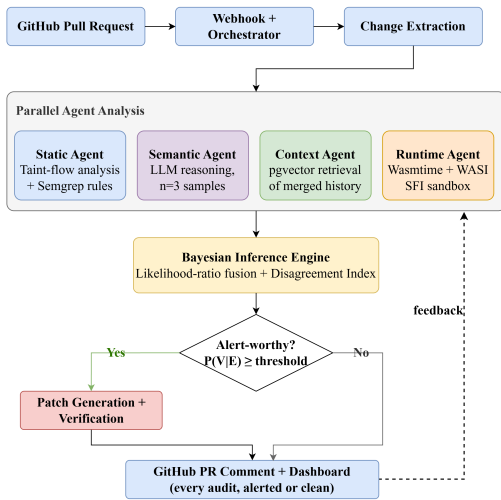


Fig. 1. Workflow of CodeSheriff.

As illustrated in Fig. 1, the prepared code is analyzed by four specialized agents: Static, Semantic, Context, and Runtime. Each agent examines the changes from a different perspective and produces structured evidence about potential

TABLE I
IMPLEMENTATION TECHNOLOGY STACK

Layer	Technology
API, queue,	FastAPI, Celery, Redis, PostgreSQL 16 with pgvector
storage	tor
Parsing	tree-sitter for extraction, ast for patch checks
Structural	Custom taint engine over networkx, Semgrep [21]
Semantic	Hosted Gemini Flash-Lite model, $n = 3$ samples
Context	Sentence-Transformers bge-small-en-v1.5, 384-dim
Runtime	Wasmtime with WASI, CPython 3.12 built to WebAssembly
Dashboard	Next.js, TypeScript, Tailwind, Recharts
Quality gates	1249 tests, mypy --strict, ruff, import-linter

vulnerabilities. The collected evidence is then passed to a Bayesian Inference Engine, which fuses the agents' likelihood ratios into a posterior probability and compares it with the fitted alert threshold to produce the final confidence-aware vulnerability verdict.

When a vulnerability is identified, the patching module generates a candidate fix, which is re-examined by the deterministic agents before it is suggested. The audit outcome, comprising the verdict, confidence score, supporting evidence and, where applicable, patch information, is returned to both the pull request and the dashboard. As depicted by the feedback loop in Fig. 1, merged code is fed back into the agent analysis stage, so the repository's own history informs the evaluation of future pull requests.

IV. IMPLEMENTATION DETAILS**A. System Architecture**

CodeSheriff is implemented in Python 3.12 as a workspace of nine packages and three applications, and runs as two processes holding different credentials. The API process owns the webhook secret and never analyses code: it verifies the X-Hub-Signature-256 header against the raw request body, writes an audit row, queues the audit identifier and answers within three seconds, inside GitHub's ten-second limit. The worker process owns the model key, fetches the changed files, runs the agents and posts the result. The queued message carries only the identifier, so a compromised queue discloses no source code. Table I lists the technology used at each layer.

B. Agent Interface and Isolation

Every agent implements exactly one function, shown in Fig. 2. An agent may import the shared contract package and nothing else: not a sibling agent, the fusion engine, the database or the labelled corpus. These boundaries are checked by import-linter in the build, so the agent independence assumed by (6) is enforced mechanically rather than by convention. An agent never raises: every failure path returns an abstention with a distinct reason, and returning an empty list is a defect, because it is indistinguishable from silence.

```
# the contract every agent implements
def analyze(unit: ChangeUnit) -> list[Evidence]:
    ...
# three statements, never an empty list
Evidence.detection(cwe, raw_score, key)
Evidence.silence(covered_cwes)
Evidence.abstention(reason)
```

Fig. 2. The agent contract. One function, three kinds of statement.

C. Sandbox Configuration

The Runtime Agent loads a pinned CPython build compiled to WebAssembly and verifies its SHA-256 digest before use; a mismatch is fatal. Capabilities are absent rather than filtered, so `os.environ` is empty because nothing was inherited, and WASI preview 1 defines no socket call, so a module asking to open a network connection fails to instantiate. Each run is bounded by fuel, wall-clock time and memory, and each unit gets a fresh store. Without the interpreter the agent abstains per unit at $\Lambda = 1$ rather than failing the audit.

D. Ground-Truth Corpus and Protocol

Calibration can only be measured against ground truth, so the evaluation uses a hand-written corpus of 76 Python functions arranged as 38 *twin pairs*: the same function once vulnerable and once fixed, which makes a false positive directly visible as a detection on the safe twin. The corpus covers ten weakness classes of the Common Weakness Enumeration (CWE): 22, 78, 79, 89, 94, 502, 639, 798, 862 and 918. Pairs, not individual cases, are assigned to splits, so twins never straddle one: 46 units for calibration, 14 for validation and 16 held out for test. The protocol is fixed in advance: ratios are fitted only on calibration, the threshold is selected only on validation, and the test split is evaluated exactly once at the end. Because every case has a twin, corpus prevalence is 0.5 by construction, which is not a property of real pull requests; the prior is therefore declared as $\pi = 0.03$ and all metrics are weighted so that vulnerable claims carry 3% of the weight.

V. RESULT ANALYSIS AND DISCUSSION

A. Experimental Setup and Metrics

Experiments were run on a system with 16 GB RAM and an Intel Core i5 processor. Evaluation replays committed model responses, so every number is reproducible without a network call. A statement about one weakness in one unit is called a *claim*: the calibration split yields 49 claims and the validation split 15, of which 7 are vulnerable.

Detection quality is reported as precision (P), recall (R) and F_1 . Reliability is reported with the Brier score [22] and the expected calibration error (ECE) [23]:

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2, \quad \text{ECE} = \sum_{b=1}^{10} \frac{|B_b|}{N} |\text{freq}_b - \text{conf}_b| \quad (8)$$

where p_i is the predicted probability of claim i , $y_i \in \{0, 1\}$ its true label, and B_b the set of claims whose probability falls

TABLE II
FITTED LIKELIHOOD RATIOS PER AGENT AND EVIDENCE CELL (–: CELL NEVER OBSERVED, CONTRIBUTES EXACTLY 1.0)

Agent	HIGH	MEDIUM	LOW	SILENCE
Structural	2.22	15.56	–	0.065
Semantic	9.48	0.56	0.74	0.304
Context	2.00	4.00	2.00	0.167
Runtime	6.92	–	–	0.115

in bin b . Both are zero for a perfect system and both must be read together, as shown below.

B. Per-Agent Detection Results

Each agent was first measured alone on the calibration split, over the cases the corpus predicts it can detect and over all safe twins. The three deterministic agents reached full recall in their own scope with no false positive: 13/13 cases for the Structural Agent over 23 safe twins, 5/5 for the Context Agent over 11 twins, and 9/9 for the Runtime Agent over 23 twins. The Semantic Agent is measured differently, because a language model fails in ways a rule cannot: none of its accepted findings quoted a sink absent from the code, no instruction injected into the analysed code changed its answer, and it correctly reported nothing on 94% of safe twins.

C. Fitted Likelihood Ratios

Table II lists the ratios fitted with (5) on the calibration split. A medium structural detection multiplies the odds of a vulnerability by 15.56, while structural silence divides them by about 15. Two cells are worth noting: a low or medium semantic detection, that is a weakness reported by only one or two of the three samples, has a ratio below 1 and is therefore evidence of *safety*. A voting scheme would count such a report as a “yes” vote; the fitted ratio records what the data shows instead.

D. Comparative Analysis

CodeSheriff is compared with two inference baselines that consume the *same* agent evidence, so the only difference is how it is combined. *Majority vote* scores a claim by the share of agents that detected, ignoring abstentions, which is what voting offers in place of a probability; *prior only* ignores the agents and always predicts the 3% base rate. All three use $\tau = 0.227$. Table III reports both splits and Fig. 3(a) plots the validation split.

CodeSheriff attains an F_1 of 0.923 at a precision of 1.000, alerting on 6 of 15 claims. Majority voting over identical evidence reduces F_1 to 0.142 and raises ECE from 0.027 to 0.141: with no prior to moderate a single agent’s alert, it finds every vulnerability but most of its ten alerts are false. The prior-only baseline reaches $\text{ECE} = 0.000$ by detecting nothing, which shows ECE alone is insufficient; its Brier score of 0.029, more than twice CodeSheriff’s 0.013, exposes the difference. The same ordering holds on the larger calibration split.

TABLE III
DETECTION AND CALIBRATION USING IDENTICAL AGENT EVIDENCE.
BEST PER COLUMN IN BOLD.

Split	System	ECE ↓	Brier ↓	P ↑	R ↑	F_1 ↑
Valid.	Prior only	0.000	0.029	0.000	0.000	0.000
	Majority vote	0.141	0.059	0.076	1.000	0.142
	CodeSheriff	0.027	0.013	1.000	0.857	0.923
Calib.	Prior only	0.000	0.029	0.000	0.000	0.000
	Majority vote	0.092	0.068	0.161	0.957	0.276
	CodeSheriff	0.009	0.010	1.000	0.696	0.821

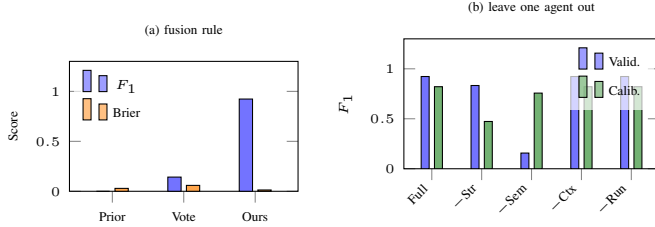


Fig. 3. (a) Detection quality and calibration error for the three fusion rules on the validation split. (b) F_1 after removing one agent at a time, on both splits.

E. Calibration Quality

Fig. 4 plots, for each probability bin on the validation split, the mean predicted probability against the observed share of vulnerable claims; a perfectly calibrated system lies on the diagonal. Most claims receive a very low probability and are indeed safe, and every high-probability claim is vulnerable. Two middle bins lie above the diagonal, so the system was slightly *under*-confident there: claims reported at 0.23 and 0.34 were vulnerable. Those bins hold one or two claims each and contribute little to the ECE; judging that region reliably requires a larger corpus.

F. Threshold Selection

The threshold was selected on the validation split by sweeping every distinct posterior value and keeping the best weighted F_1 . Below 0.227 recall is complete, but one safe claim at 0.13 is alerted, and because safe code dominates in production that single false alarm drives weighted precision to 0.175 and F_1 to 0.291. Above 0.227 precision stays at 1.000 while recall falls to 0.571 at $\tau = 0.345$ and 0.286 at $\tau = 0.969$, so $\tau = 0.227$ is the balance point. These figures are selection-time estimates, not held-out measurements.

G. Ablation Study

Each agent was removed in turn to isolate its contribution. A removed agent contributes an abstention at likelihood ratio 1.0, so the posterior changes only through the evidence it no longer supplies. Table IV reports both splits, and Fig. 3(b) plots the resulting F_1 .

The two splits disagree about which agent matters most, and that disagreement is itself the main finding. On validation, removing the Semantic Agent costs 0.766 in F_1 and collapses

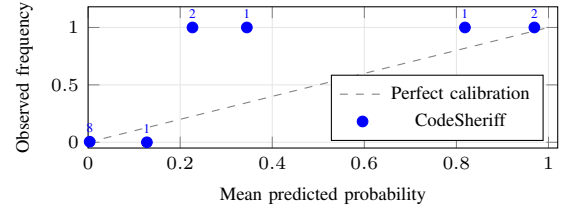


Fig. 4. Reliability diagram on the validation split. Labels give the number of claims in each bin.

TABLE IV
LEAVE-ONE-AGENT-OUT ABLATION AT $\tau = 0.227$

Configuration	Validation split			Calibration split		
	ECE	Brier	F_1	ECE	Brier	F_1
Full model	0.027	0.013	0.923	0.009	0.010	0.821
– Structural	0.012	0.015	0.833	0.007	0.017	0.473
– Semantic	0.047	0.031	0.157	0.011	0.015	0.757
– Context	0.026	0.013	0.923	0.007	0.011	0.821
– Runtime	0.028	0.013	0.923	0.011	0.011	0.821

precision to 0.096; on calibration, removing the Structural Agent is the most damaging, costing 0.348 in F_1 as precision falls to 0.359, while the Semantic Agent costs only 0.064 there. No single agent is sufficient on both splits, which is the empirical case for heterogeneous agents. Removing the Context or Runtime Agent leaves every decision unchanged; this is a null result rather than redundancy, since both have deliberately narrow scopes and usually confirm a decision another agent has already reached, while still moving the posterior: calibration ECE rises from 0.009 to 0.011 without the Runtime Agent.

H. Comparison with Related Systems

A direct numerical comparison with published systems is not meaningful, because they are evaluated on different languages, weakness sets and metric protocols. MAVUL [18], the closest multi-agent precedent, targets C/C++ memory-safety CWEs under its own P-C/P-V/P-B/P-R protocol and reports no reliability measure; DeepSecure [5] is a Python-specific hybrid deep-learning detector with a confidence component but no repository context or execution evidence; AegisGuard [14] adds context and risk stratification; and Reddy *et al.* [19] triage with knowledge-based agents. Table V therefore compares capabilities rather than scores. To our knowledge, CodeSheriff is the first of these to combine four different techniques with a calibrated probability, sandboxed execution and verified patch suggestions inside the pull-request workflow.

I. Assumptions and Limitations

The corpus is Python-only, self-authored and covers a closed set of ten CWEs, so the reported rates describe this corpus and not arbitrary pull requests. The validation split, at 15 claims, is too small for a meaningful confidence interval, and the threshold was selected on it; the held-out test split

TABLE V
CAPABILITY COMPARISON WITH RELATED SYSTEMS (✓: REPORTED; −:
NOT REPORTED IN THE CITED WORK)

Capability	[5]	[14]	[18]	[19]	Ours
Multiple agents	−	−	✓	✓	✓
Distinct technique per agent	−	−	−	−	✓
Calibrated confidence	✓	−	−	−	✓
Repository context	−	✓	✓	−	✓
Sandboxed execution	−	−	−	−	✓
Verified patch in the PR	−	−	−	−	✓

remains unevaluated. Evaluation is a single deterministic run against committed model recordings, so no sampling variance is reported. Semgrep has no build for the fitting platform, so the structural ratios in Table II describe a one-backend agent. Finally, the 3% base rate is declared rather than measured.

VI. CONCLUSION AND FUTURE SCOPE

A. Conclusion

This paper presented CodeSheriff, a heterogeneous multi-agent security auditor for GitHub pull requests that fuses structural, semantic, contextual and runtime evidence through Bayesian inference rather than majority voting. The design keeps the four agents independent, separates an agent that found nothing from one that could not run, and fits every likelihood ratio and the alert threshold on labelled data. On the validation split it attained an F_1 of 0.923 at a precision of 1.000 with an expected calibration error of 0.027, outperforming a majority-vote combiner over identical evidence by a factor of 6.5 in F_1 while reducing calibration error five-fold, and the ablation showed that no single agent accounts for this. Unlike existing detectors, which report a verdict with no reliability measure, its central contribution is a posterior probability that is simultaneously discriminative and calibrated, verified jointly through F_1 and the Brier score, and accompanied by patch suggestions that pass a fixed verification ladder.

B. Future Scope

Several directions follow from Section V-I. The structural agent must be re-observed on a Linux or CI environment where both of its backends are available, since the current fit describes a one-backend agent, and the held-out test split should then be scored exactly once under the protocol of Section IV. Scoring Semgrep, Bandit and a single-pass LLM under the same weighted protocol would complete the external comparison, and enlarging the corpus with real vulnerability-fixing commits would allow interval estimation and a sensitivity analysis over the threshold and the declared base rate. Finally, extending language coverage to JavaScript and TypeScript, which the tree-sitter based extraction already permits, evaluating a fine-tuned semantic model against the hosted one, and using the same sandbox to release CI/CD secrets only to verified code are identified as future work.

REFERENCES

- [1] G. Li and Y. Yang, “On the code vulnerability detection based on deep learning: A comparative study,” *IEEE Access*, 2024.
- [2] S. Han, H. Nam, J. Kang, K. Kim, S. Cho, and S. Lee, “Code-smash: Source-code vulnerability detection using siamese and multi-level neural architecture,” *IEEE Access*, 2024.
- [3] S. Han, M. Kim, J. Kang, K. Kim, S. Lee, and S. Lee, “Similarity-based source code vulnerability detection leveraging transformer architecture: Harnessing cross-attention for hierarchical analysis,” *IEEE Access*, 2024.
- [4] B. K. E. Abbasi and A. Shameli-Sendi, “Leveraging key source code metrics for predicting software vulnerabilities with machine learning,” *IEEE Access*, 2026.
- [5] R. A. Almutairi and N. Abbadeni, “Deepsecure: Hybrid deep learning framework for detecting security vulnerabilities in python code,” *IEEE Access*, 2026.
- [6] A. Shestov, R. Levichev, R. Mussabayev, E. Maslov, P. Zadorozhny, A. Cheshkov, R. Mussabayev, A. Toleu, G. Tolegen, and A. Krassovitskiy, “Finetuning large language models for vulnerability detection,” *IEEE Access*, 2025.
- [7] K. Tamberg and H. Bahsi, “Harnessing large language models for software vulnerability detection: A comprehensive benchmarking study,” *IEEE Access*, 2025.
- [8] D. Gnieciak and T. Szandala, “Large language models versus static code analysis tools: A systematic benchmark for vulnerability detection,” *IEEE Access*, 2025.
- [9] B. Gülmöz, “Code generation with large language models: a survey from neural program synthesis to autonomous software development,” *Applied Intelligence*, vol. 56, no. 6, p. 200, 2026.
- [10] Y. Chen, Y. Shen, T. Wang, S. Ou, R. Wang, Y. Li, and Z. Pan, “Software defect detection using large language models: a literature review,” *Frontiers of Computer Science*, vol. 20, no. 6, p. 2006202, 2026.
- [11] H. Su, Z. Xu, Y. Zhang, and Q. Tan, “Source code vulnerability detection based on deep learning: a review,” *Cybersecurity*, vol. 9, no. 1, p. 2, 2026.
- [12] L. B. Germano, R. R. Goldschmidt, R. C. Noya, and J. C. Duarte, “A systematic review on detection, repair, and explanation of vulnerabilities in source code using large language models,” *IEEE Access*, 2025.
- [13] G. Antal, L. Értekes, N. Szolnoki, and P. Hegedűs, “Evaluating retrieval-augmented generation for llm-based vulnerability detection: An empirical study on real-world java vulnerabilities,” *IEEE Access*, 2026.
- [14] P.-C. Huang, C.-H. Mao, and C.-M. Lai, “Aegisguard: A context-aware framework for semantic vulnerability detection and risk stratification,” *IEEE Access*, 2026.
- [15] H. Gao, D. Zhang, and L. Chen, “Scout: Llm-driven vulnerability detection via interactive repository exploration,” *IEEE Access*, 2026.
- [16] I. S. Göçmen, A. S. Cezayir, and E. Tüzün, “Enhanced code reviews using pull request based change impact analysis,” *Empirical Software Engineering*, vol. 30, no. 3, p. 64, 2025.
- [17] Z. Wei, J. Sun, Y. Sun, Y. Liu, D. Wu, Z. Zhang, X. Zhang, M. Li, Y. Liu, C. Li, M. Wan, J. Dong, and L. Zhu, “Advanced smart contract vulnerability detection via llm-powered multi-agent systems,” *IEEE Transactions on Software Engineering*, 2025.
- [18] Y. Li, K. Joshi, X. Wang, and E. Wong, “Mavul: Multi-agent vulnerability detection via contextual reasoning and interactive refinement,” in *2025 IEEE 7th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*, 2025.
- [19] K. P. Reddy, J. S. Dhatteval, et al., “Agentic artificial intelligence with knowledge-based reasoning for automated vulnerability triage in collaborative software repositories,” in *2026 IEEE 6th International Conference on Computing, Power, and Communication Technologies (IC2PCT)*. IEEE, 2026, pp. 493–498.
- [20] T. A. Syed, M. Riyaz Belgaum, S. Jan, A. A. Khan, and S. S. Alqahtani, “Agentic ai for autonomous defense in software supply chain security: Beyond provenance to vulnerability mitigation,” in *2025 International Conference on Computer and Applications (ICCA)*, 2025.
- [21] Semgrep, Inc., “Semgrep: Lightweight static analysis for many languages,” <https://semgrep.dev>, accessed: Sep. 2026.
- [22] G. W. Brier, “Verification of forecasts expressed in terms of probability,” *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.
- [23] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330.